```
fn main() -> i32 {
    println("Hello, Amiga!")
    return 0
}
```

NOVUS

Programmer's Reference Guide

New code for classic machines

Version 0.1 | December 2025

For Amiga 68020+ | AmigaOS 2.0+

Contents

Foreword

The Amiga was ahead of its time. In 1985, while other personal computers offered beeps and static screens, the Amiga delivered pre-emptive multitasking, dedicated graphics and audio hardware, and a sophisticated operating system that wouldn't be matched by mainstream competitors for nearly a decade.

For many of us, the Amiga wasn't just a computer—it was where we learned to code, to create, to push hardware to its limits. The demoscene flourished. Games achieved visual and audio fidelity that seemed impossible. A generation of programmers cut their teeth on 68000 assembly, crafting tight loops that squeezed every cycle from the machine.

Then the platform faded from the mainstream. But it never truly died.

Today, a vibrant community keeps the Amiga alive. Original hardware is lovingly maintained and expanded. FPGA recreations achieve cycle-perfect accuracy. Emulation lets new generations experience what made these machines special. And remarkably, people still write software for them—not out of obligation, but out of genuine passion.

Yet the tools available to modern Amiga developers are largely the same ones we used thirty years ago. C compilers like VBCC and GCC produce excellent code, but the languages themselves haven't evolved. The patterns and safety mechanisms that modern systems programming takes for granted—sum types, pattern matching, resource management without garbage collection—remain out of reach.

Novus aims to change that.

This is not an attempt to modernize the Amiga by hiding what makes it special. Quite the opposite. Novus is designed to *embrace* the Amiga's architecture: its custom chips, its message-passing OS, its elegant hardware. The goal is to give developers modern ergonomics and safety while preserving—even enhancing—the direct hardware access that makes Amiga programming rewarding.

Whether you're a veteran who remembers waiting for Workbench to load from floppy, or a newcomer curious about this legendary platform, we hope Novus helps you write code that's safer, clearer, and more enjoyable—without sacrificing an ounce of the control that Amiga development demands.

New code for classic machines. Welcome to Novus.

December 2025

Chapter 1

Introduction

1.1 What is Novus?

Novus is a systems programming language designed specifically for the Amiga 68k platform. It compiles to native 68020+ machine code and integrates deeply with AmigaOS, producing executables that run on real hardware and accurate emulators alike.

The name comes from the Latin word for "new"—reflecting the project's mission to bring modern language design to classic machines. Novus is not a port of an existing language, nor is it a lowest-common-denominator cross-platform tool. Every design decision is made with the Amiga in mind.

1.1.1 Key Characteristics

Native Performance Novus compiles directly to 68k assembly, then to Amiga executables via the VBCC toolchain. There is no interpreter, no virtual machine, no runtime overhead beyond what you explicitly request.

Modern Safety The type system catches errors at compile time. `Result` and `Option` types make error handling explicit. Resource cleanup is deterministic via `defer` blocks. You get safety without garbage collection.

Direct Hardware Access Novus doesn't hide the Amiga's hardware behind abstractions you can't escape. Custom chip registers, Exec library calls, copper lists, blitter operations—all are first-class citizens with typed, safe interfaces.

Readable Output The generated assembly is clean and well-commented. When you need to understand what the compiler is doing—and on the Amiga, you often do—the output won't fight you.

Amiga-Native Async The `async`/`await` model maps directly to AmigaOS primitives: Exec signals and message ports. No hidden threads, no runtime scheduler—just the OS facilities the Amiga provides.

1.2 Why Novus Exists

The Amiga platform has capable C compilers. VBCC, in particular, generates excellent code and remains actively maintained. So why create a new language?

1.2.1 The Limits of C

C was revolutionary in 1972. It remains useful today precisely because it's minimal—a portable assembler that gets out of your way. But that minimalism has costs:

- **No sum types.** Representing "this value is either an error or a result" requires conventions that the compiler can't enforce.

- **No built-in error handling.** Return codes are easily ignored. Exceptions don't exist. Every function call is an opportunity for silent failure.
- **Manual resource management.** Matching every `AllocMem` with a `FreeMem`, every `OpenLibrary` with a `CloseLibrary`, every `Lock` with an `UnLock`—across all control flow paths, including error cases.
- **Primitive module system.** Header files and `#include` are textual substitution. Name collisions, include order dependencies, and compile time explosion are facts of life.
- **No metaprogramming.** The preprocessor is powerful but crude. Anything beyond simple macros quickly becomes write-only code.

These aren't theoretical concerns. They're the source of real bugs in real Amiga software—memory leaks, use-after-free, null pointer crashes, resource exhaustion from unclosed libraries.

1.2.2 Modern Solutions, Amiga Constraints

Languages like Rust address these problems elegantly. But Rust targets modern systems with megabytes of RAM, gigahertz CPUs, and operating systems that provide virtual memory and threads. Its runtime assumptions don't map to the Amiga.

Novus takes a different approach: provide modern ergonomics within Amiga constraints.

- `Result[T, E]` and `Option[T]` are zero-cost at runtime—they're just tagged unions that the compiler understands.
- `defer` blocks compile to straightforward cleanup code inserted at scope exit. No hidden allocations, no unwinding tables.
- Pattern matching compiles to efficient jump tables or comparison chains as appropriate.
- The module system is compile-time only. No runtime symbol resolution, no dynamic linking overhead.
- Fixed-point math uses inline 68020+ instructions, not floating-point emulation.

The result is a language that *feels* modern but *runs* like hand-tuned assembly.

1.3 Design Philosophy

Novus follows several core principles that inform every language decision:

1.3.1 Explicit Over Implicit

If something allocates memory, the code should say so. If something can fail, the type should reflect it. If a function has side effects, they should be visible at the call site.

This doesn't mean verbose—Novus has type inference, sensible defaults, and concise syntax. But it means no hidden costs, no action at a distance, no "magic" that obscures what the machine is actually doing.

1.3.2 Predictable Performance

On a 7 MHz 68000, every cycle matters. On a 50 MHz 68060, you have more headroom—but you still can't afford hidden allocations in a tight loop or cache-hostile memory access patterns.

Novus gives you control. The compiler won't secretly box values, won't insert invisible runtime checks in release builds, won't generate code that's impossible to reason about.

1.3.3 Respect the Machine

The Amiga isn't a Unix workstation. It has custom chips that do things no other hardware can do. Its operating system uses message passing and signals, not files and processes. Its memory model is flat and physical.

Novus doesn't pretend otherwise. The standard library wraps AmigaOS idiomatically—`Result` types for operations that can fail, handles for resources that need cleanup—but it doesn't try to be POSIX. Amiga code should look like Amiga code.

1.3.4 Fail Fast, Fail Clearly

When something goes wrong, you should know immediately and know exactly what happened. Compile-time errors are better than runtime errors. Runtime errors with clear messages are better than silent corruption.

In debug builds, Novus checks array bounds, validates pointers, and panics with meaningful diagnostics. In release builds, those checks can be removed—but the type system still prevents the errors that checks would catch.

1.4 Target Audience

This guide assumes you're a working programmer. You should be comfortable with:

- Systems programming concepts: pointers, memory allocation, stack vs. heap
- Basic 68k architecture: registers, addressing modes, the supervisor/user split
- AmigaOS fundamentals: libraries, Exec, DOS, the message-passing model

Prior experience with Rust, Swift, or other modern systems languages will make Novus feel familiar, but it's not required. If you've written C for the Amiga, you have all the background you need.

1.5 How to Read This Guide

The guide is organized in layers:

Part I: Language Fundamentals covers the core language—syntax, types, control flow, functions, and modules. Read this first.

Part II: Standard Library documents the `std` packages: collections, strings, memory management, and Amiga OS bindings.

Part III: Advanced Topics covers async programming, inline assembly, FFI with existing C code, and performance optimization.

Appendices provide quick references, grammar summaries, and migration guides from C.

Code examples are designed to be complete and runnable. When a snippet requires context, we provide it. When we omit boilerplate, we say so.

Let's begin.

Chapter 2

Getting Started

> *"The journey of a thousand miles begins with a single step."*
> —Lao Tzu

This chapter walks you through installing the Novus compiler, writing your first program, and understanding the compilation pipeline. By the end, you'll have a working executable running on your Amiga (or emulator).

2.1 Prerequisites

Before installing Novus, ensure you have:

- **.NET 9.0 or later** — The Novus compiler is written in C# and requires the .NET runtime. Download from `https://dotnet.microsoft.com`
- **VBCC toolchain** — Novus uses VBCC's assembler (`vasmm68k_mot`) and linker (`vlink`) to produce Amiga executables. The toolchain is vendored in the repository under `vendor/vbcc`, or you can specify a custom installation via the `-vbcc-path` option.
- **An Amiga or emulator** — To run your compiled programs. FS-UAE, WinUAE, or real hardware all work. A 68020+ configuration with AmigaOS 2.0 or later is required.

2.2 Installation

2.2.1 Building from Source

Clone the repository and build:

```
git clone https://github.com/your-repo/novus.git
cd novus
dotnet build
```

The compiler will be available at `Novus/bin/Debug/net9.0/novus`.
For a release build with optimizations:

```
dotnet build -c Release
```

2.2.2 Verifying Installation

Check that the compiler runs:

```
./Novus/bin/Debug/net9.0/novus --help
```

You should see the available commands and options.

2.3 Your First Program

Create a file named `hello.novus`:

```
from std::io::file import println

fn main() -> i32 {
    println("Hello, Amiga!")
    return 0
}
```

Let's break this down:

- `from std::io::file import println` — Imports the `println` function from the standard library. Unlike some languages, Novus requires explicit imports.
- `fn main() -> i32` — Declares the entry point. All Novus programs must have a `main` function that returns an `i32` (the exit code).
- `println("Hello, Amiga!")` — Prints the message to standard output.
- `return 0` — Returns success (zero) to the operating system.

2.3.1 Compiling

Compile to an Amiga executable:

```
novus compile hello.novus -o hello
```

This produces an executable named `hello` in Amiga HUNK format. Without the `-o` flag, the output would default to `a.out`.

2.3.2 Running on the Amiga

Transfer the executable to your Amiga (via shared folder, network, or disk image) and run it from the Shell:

```
1.Workbench:> hello
Hello, Amiga!
```

Congratulations—you've run your first Novus program!

2.4 Understanding the Toolchain

The Novus compilation pipeline has several stages:

1. **Parsing** — Source code is parsed into an Abstract Syntax Tree (AST)
2. **Semantic Analysis** — Types are checked, names are resolved, and the Intermediate Representation (IR) is built
3. **C Code Generation** — The IR is lowered to C99 code targeting VBCC
4. **Compilation** — VBCC's C compiler (`vc`) compiles the C code directly to 68k object files

5. **Assembly** — Additional assembly files (runtime, library stubs) are assembled with `vasm`
6. **Linking** — `vlink` combines all object files with the runtime and produces the final executable

This pipeline means you get the benefits of VBCC's mature 68k code generation while writing modern, safe Novus code.

2.4.1 Compiler Commands

Novus provides several commands:

`novus compile` Compile a single source file to an executable

`novus build` Build a project defined by `project.toml`

`novus test` Discover and compile tests

`novus new` Create a new project from a template

`novus fmt` Format source code

`novus clean` Remove cached build artifacts

2.4.2 Compile Options

Common options for `novus compile`:

`-o, -output` Output file name (default: `a.out`)

`-cpu` Target CPU: `68020`, `68030`, `68040`, `68060`, or `auto` (default: `auto`)

`-fpu` FPU mode: `none`, `68881`, `68882`, `68040`, `68060`, or `auto` (default: `auto`)

`-release` Enable optimizations and disable debug checks

`-O` Optimization level (0–2)

`-emit-asm` Emit assembly output only; do not assemble or link

`-emit-ir` Emit IR to stdout for debugging

`-v, -verbose` Show detailed compilation progress

The `auto` CPU setting produces a "fat binary" that detects the CPU at runtime and uses optimized code paths accordingly.

2.4.3 Debug vs Release Builds

By default, Novus compiles in debug mode:

- Array bounds checking is enabled
- Debug symbols are included
- Optimizations are minimal

For production builds, use `-release`:

```
novus compile hello.novus -o hello --release
```

Release builds are smaller and faster, but runtime errors provide less diagnostic information.

Safety Levels

For finer control, use `-safety-level`:

Level 0 Unsafe — no runtime checks (use with `-unsafe` flag)

Level 1 Basic — minimal checks

Level 2 Full — standard safety checks (default for debug)

Level 3 Paranoid — maximum checking

2.5 Project Structure

For anything beyond a single file, create a project with `novus new`:

```
novus new myproject --template cli
cd myproject
```

This creates a standard project structure:

```
myproject/
    project.toml        # Project configuration
    src/
        main.novus      # Entry point
```

2.5.1 The project.toml File

The `project.toml` file defines project metadata and build settings:

```
[package]
name = "myproject"
version = "0.1.0"

[build]
target_cpu = "68020"
fpu = "none"
```

Available build settings include:

`target_cpu` Target CPU (68020, 68030, 68040, 68060, or auto)

`fpu` FPU mode

`chipset` Target chipset (ocs, ecs, aga, or auto)

`optimization_level` Optimization level (0–2)

`output` Output file name

Build the project with:

```
novus build
```

2.6 The Standard Library

Novus includes a standard library providing common functionality:

std::core Fundamental types: `Option`, `Result`, and core traits

std::collections Data structures: `Vec`, `HashMap`, `HashSet`, `VecDeque`, `RingBuffer`, `SlotMap`, and more

std::strings String handling: `String`, `Str`, `StringBuilder`

std::memory Memory management: `MemoryBlock`, chip memory pools, slices

std::math Fixed-point arithmetic, trigonometry, vectors, easing functions

std::io Input/output: `println`, file operations

std::ffi AmigaOS bindings: Exec, DOS, Graphics, Intuition, and more

Import modules with `from`:

```
from std::collections::vec import Vec
from std::io::file import println

fn main() -> i32 {
    var names: Vec<&str> = Vec::new()
    names.push("Amiga")
    names.push("Novus")

    println("Count: {}", names.len())
    return 0
}
```

2.7 Running Tests

Novus has built-in test support. Mark test functions with the `@test` attribute:

```
from std::test::test import expect_eq

@test
fn test_addition() {
    expect_eq(2 + 2, 4, "basic addition")
}

@test
fn test_multiplication() {
    expect_eq(3 * 7, 21, "basic multiplication")
}
```

The `expect_eq` function uses overloading to work with any comparable type—no need for type-specific variants like `expect_eq_i32`.

Compile the test runner with:

```
novus test mytest.novus
```

This discovers all `@test` functions and compiles a test executable. The output tells you where the executable was created:

```
Test runner built successfully!
Output: /path/to/tests
Tests: 2 active, 0 skipped

Copy to Amiga and run to execute tests.
```

Important: The test command builds the test runner but does not execute it. You must transfer the executable to your Amiga and run it there to see test results.

2.7.1 Test Options

The `novus test` command supports several options:

`-filter` Run only tests matching a pattern

`-list` List discovered tests without building

`-b, -benchmark` Include timing information in output

`-output-dir` Specify where to write the test executable

2.8 Next Steps

You now have a working Novus installation and understand the basic workflow. The following chapters cover:

- **Chapter 3: Language Basics** — Variables, types, and expressions
- **Chapter 4: Types** — The type system in depth
- **Chapter 5: Control Flow** — Conditionals, loops, and pattern matching

When you're ready, turn the page and start learning the language itself.